

Certified program synthesis with a multi-modal verifier



Yueyang
Feng^{1,*}



Dipesh
Kafle^{1,*}



Vladimir
Gladshstein¹



Vitaly
Kurin²



George
Pîrlea¹



Qiyuan
Zhao¹



Peter
Müller³



Ilya
Sergey¹

¹National University of Singapore ²Neapolis University Pafos ³ETH Zurich

*Joint first authors

ASE '26 · October 15, 2026 · Munich, Germany

AI writes code for us. An idea becomes an implementation in seconds.

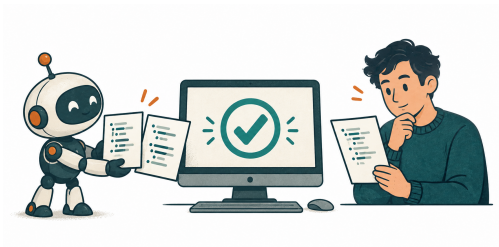


AI writes code for us. An idea becomes an implementation in seconds.



Who reviews all this code?

Generating code is fast. Establishing confidence still takes tremendous amount of work.



Ask the AI for a proof, too.

Fermat's Last Theorem

13 million lines of Lean;
machine-checked proof.

[Anthropic, September 2026](#)

Navier-Stokes

Proof released with a
Lean formalization.

[OpenAI, September 2026](#)



Ask the AI for a proof, too.

Fermat's Last Theorem

13 million lines of Lean;
machine-checked proof.

[Anthropic, September 2026](#)

Navier–Stokes

Proof released with a
Lean formalization.

[OpenAI, September 2026](#)

In an ideal world:

User gives a task description;

AI gives a specification + implementation + proof.

Our goal: Certified synthesis from natural language

Generate a runnable program and a checked correctness proof from an English task.

Our goal: Certified synthesis from natural language

Generate a runnable program and a checked correctness proof from an English task.

Two key challenges

Assessing specifications effectively

Does the specification capture
the intended task?

Our goal: Certified synthesis from natural language

Generate a runnable program and a checked correctness proof from an English task.

Two key challenges

Assessing specifications effectively

Does the specification capture the intended task?

Constructing proofs efficiently

Can we complete the proof within a limited budget?



The idea

Multi-modal
verification



The idea

Multi-modal
verification



The pipeline

Inside LeetProof



Our approach: Multi-modal verification

Our approach: Multi-modal verification

Test

Check examples with unit tests;
search for counterexamples
with PBT.

Automate

Discharge goals with SMT and
Lean automation like **grind**.

Prove interactively

Complete proofs when
automation gets stuck.

Our approach: Multi-modal verification

Test

Check examples with unit tests;
search for counterexamples
with PBT.

Automate

Discharge goals with SMT and
Lean automation like **grind**.

Prove interactively

Complete proofs when
automation gets stuck.

Velvet combines these *modes* for imperative programs in Lean.

Our approach: Multi-modal verification

Test

Check examples with unit tests;
search for counterexamples
with PBT.

Automate

Discharge goals with SMT and
Lean automation like **grind**.

Prove interactively

Complete proofs when
automation gets stuck.

Velvet combines these *modes* for imperative programs in Lean.

We present **LeetProof**, which uses multi-modality to turn English tasks into verified programs.

```

method IsNonPrime (n : Nat) return (result : Bool)
  ensures result = true  $\leftrightarrow$   $\neg$ isPrime n
  do
    if n  $\leq$  1 then return true
    let mut i : Nat := 2
    let mut ret : Bool := false
    while i * i  $\leq$  n
      invariant  $\neg$ ret  $\leftrightarrow$   $\forall d, 2 \leq d \wedge d < i \rightarrow n \% d \neq 0$ 
      invariant i  $\geq$  2  $\wedge$  (i - 1) * (i - 1)  $\leq$  n
    do
      if n % i = 0 then ret := true
      i := i + 1
    return ret

prove_correct IsNonPrime by
  loom_solve -- automated VC discharge
  ...      -- remaining interactive proof

```

```
method IsNonPrime (n : Nat) return (result : Bool)
```

```
  ensures result = true  $\leftrightarrow$   $\neg$ isPrime n
```

```
  do
```

```
    if n  $\leq$  1 then return true
```

```
    let mut i : Nat := 2
```

```
    let mut ret : Bool := false
```

```
    while i * i  $\leq$  n
```

```
      invariant  $\neg$ ret  $\leftrightarrow$   $\forall d, 2 \leq d \wedge d < i \rightarrow n \% d \neq 0$ 
```

```
      invariant i  $\geq$  2  $\wedge$  (i - 1) * (i - 1)  $\leq$  n
```

```
    do
```

```
      if n % i = 0 then ret := true
```

```
      i := i + 1
```

```
    return ret
```

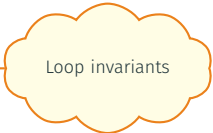
```
prove_correct IsNonPrime by
```

```
  loom_solve -- automated VC discharge
```

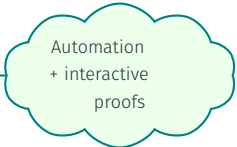
```
  ...      -- remaining interactive proof
```



Postcondition



Loop invariants



Automation
+ interactive
proofs

LeetProof orchestrates the entire pipeline from natural language to a verified implementation in the following phases:

1. Generate specification

Capture the intent of an English description in a Lean specification.



2. Synthesize program

Plain Lean: Generate a functional implementation.

Velvet: Generate an imperative implementation, then infer and annotate loops with invariants.



3. Verify the program

Prove that the synthesized program satisfies the specification.

LeetProof orchestrates the entire pipeline from natural language to a verified implementation in the following phases:

1. Generate specification

Capture the intent of an English description in a Lean specification.



2. Synthesize program

Plain Lean: Generate a functional implementation.

Velvet: Generate an imperative implementation, then **infer and annotate loops with invariants**.



3. Verify the program

Prove that the synthesized program satisfies the specification.

- Capturing intent is difficult for LLMs, and even humans.

- Capturing intent is difficult for LLMs, and even humans.
- Intent capture cannot be guaranteed, but we can effectively search for mismatches.

Step 1: Check the LLM's understanding

Task description  Formal specification + example inputs and outputs

Reuse examples from the problem statement and ask the LLM for more based on its interpretation (about ten in total).

Input	Expected output
[4, 1, 2, 3]	true
[2, 1, 3, 4]	false
[1, 1, 1]	true

Step 2: Gaining confidence in the generated specification

Step 2: Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Step 2: Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Step 2: Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Step 2: Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Expected output

Does the intended answer satisfy the postcondition?

Step 2: Gaining confidence in the generated specification

We can prove that the examples satisfy the precondition and postcondition. But, proving each example can be expensive.

Key insight: Search for counterexamples with property-based testing (PBT).

Input validity

Does the input satisfy the precondition?

Expected output

Does the intended answer satisfy the postcondition?

Output uniqueness

Does the specification also admit a different answer?

Keep the input fixed; sample alternative outputs by mutating the expected output.

Task: Return whether the array is sorted and rotated.

Candidate specification 1

```
def postcondition (nums : Array Int) (result : Bool) :=  
  result = true → rotSortedProp nums
```

Task: Return whether the array is sorted and rotated.

Candidate specification 1

```
def postcondition (nums : Array Int) (result : Bool) :=  
  result = true → rotSortedProp nums
```

With $\rightarrow$, always returning **false** satisfies the specification.

Output uniqueness test

[1, 2, 3]: **true** $\rightarrow$ **false** postcondition still holds!

Rejected by PBT

Task: Return whether the array is sorted and rotated.

Candidate specification 2

```
def postcondition (nums : Array Int) (result : Bool) :=  
  result = true  $\leftrightarrow$  rotSortedProp nums
```

Approved by pipeline

Step 1: Generate candidate invariants

Program

```
while i < n  
do  
  ...
```

Step 1: Generate candidate invariants

Program

```
while i < n  
do  
  ...
```

LLM proposes
annotations



Annotated program

```
while i < n  
  invariant "inv_bounds" ( $i \leq n$ )  
  invariant "inv_drops_count" (...)  
  decreasing n - i  
do  
  ...
```

Step 2: Validate invariants (automation)

Discharge obligations automatically using SMT / Lean's automation.

A simplified obligation for illustrative purpose

$i \ n : \text{Nat}$

$h : i < n$

$\vdash i + 1 \leq n \wedge n - (i + 1) < n - i$

Step 2: Validate invariants (automation)

Discharge obligations automatically using SMT / Lean's automation.

A simplified obligation for illustrative purpose

```
i n : Nat
h : i < n
⊢ i + 1 ≤ n ∧ n - (i + 1) < n - i
```

For the final program of our running example, 14/18 VCs were discharged using automation.

Step 2: Validate invariants (counterexample search)

A candidate invariant, with `=` changed to `<`.

```
invariant "inv_drops_count" (drops = < (Finset.filter  
  (fun k : Nat => nums[(k + 1) % n]! < nums[k]!)  
  (Finset.range i)).card)
```

Step 2: Validate invariants (counterexample search)

A candidate invariant, with `=` changed to `<`.

```
invariant "inv_drops_count" (drops = < (Finset.filter  
  (fun k : Nat => nums[(k + 1) % n]! < nums[k]!)  
  (Finset.range i)).card)
```

```
[velvet_plausible_test] FAIL:  
invariant "inv_drops_count" doesn't hold  
nums = #[-18, 10, -13, 11, 8, 17, 13, -19, 15, -1, -27, -25]
```

Generating specifications from English tasks

97.4%

Equivalent or defensible*

188 VERINA problems

150 Equivalent to reference

33 Defensible differences

* Defensible: justified differences due to reference errors, valid precondition variations, or ambiguous descriptions.

PBT uncovers issues in human-written benchmarks

VERINA

VERINA: BENCHMARKING VERIFIABLE CODE GENERATION

13 issues

CLEVER

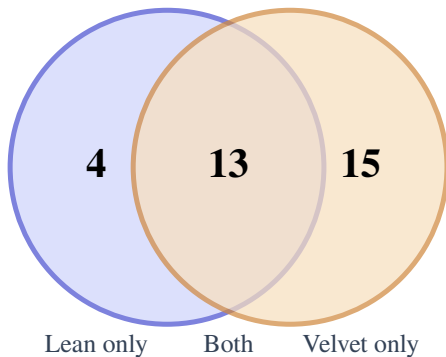
CLEVER: A Curated Benchmark for Formally Verified Code Generation

18 issues

Found without LLM assistance.

More certified solutions with Velvet compared to functional Lean, at the same cost

Fully proven (GPT-5.2): Lean 17/50; Velvet 28/50.



RQ3: Closing the remaining proofs

Can additional proof effort close the remaining goals?

Partially proven programs completed by Aristotle:

Lean 23 / 30

Velvet 18 / 18

RQ3: Closing the remaining proofs

Can additional proof effort close the remaining goals?

Partially proven programs completed by Aristotle:

Lean	23 / 30
------	---------

Velvet	18 / 18
--------	---------

RQ4: Across different LLMs

Does the advantage hold with another model?

Fully proven programs at \$5 per problem:

	Lean	Velvet
GPT-5.2	17/50	28/50
Opus 4.6	7/25*	10/25*

*25 problems chosen at random from the 50 evaluated with GPT-5.2.

Summary

What have we seen

Summary

What have we seen

- Certified synthesis needs both **faithful specifications** and **complete proofs**.

Summary

What have we seen

- Certified synthesis needs both **faithful specifications and complete proofs**.
- Testing **rejects bad candidates** before we spend effort proving them.

What have we seen

- Certified synthesis needs both **faithful specifications and complete proofs**.
- Testing **rejects bad candidates** before we spend effort proving them.
- LeetProof uses Velvet's multiple modes to **certify more programs within the same budget**.

Summary

What have we seen

- Certified synthesis needs both **faithful specifications and complete proofs**.
- Testing **rejects bad candidates** before we spend effort proving them.
- LeetProof uses Velvet's multiple modes to **certify more programs within the same budget**.

Interesting future directions

- Certified synthesis of protocols and distributed systems.

What have we seen

- Certified synthesis needs both **faithful specifications and complete proofs**.
- Testing **rejects bad candidates** before we spend effort proving them.
- LeetProof uses Velvet's multiple modes to **certify more programs within the same budget**.

Interesting future directions

- Certified synthesis of protocols and distributed systems.
- Going Beyond functional correctness.

Summary

What have we seen

- Certified synthesis needs both **faithful specifications** and **complete proofs**.
- Testing **rejects bad candidates** before we spend effort proving them.
- LeetProof uses Velvet's multiple modes to **certify more programs within the same budget**.



Verse Lab



Velvet



Artifact



Paper

Interesting future directions

- Certified synthesis of protocols and distributed systems.
- Going Beyond functional correctness.

Thank you!